

Составление простых запросов

Для составления простых запросов обычно хватает следующих разделов:
SELECT, FROM, WHERE.

```
SELECT [DISTINCT]
--имена столбцов,
--функции,
--выражение,
--подзапрос,
--* => выбираются все столбцы
FROM имя_таблицы
WHERE предикат

-- DISTINCT => выбор уникальных строк
```

Псевдоним столбца для отображения:

```
имя as псевдоним
```

Примеры простых запросов

Пример таблицы для составления запросов (имя таблицы - workers):

worker_id	last_name	salary	department_id	h.d
101	Иванов	50000	90	01.09.2000
102	Петров	30000	80	01.09.2001
103	Сидоров	40000	70	01.02.2025
104	Иванова	NULL	79	22.09.2025

Пример запроса (1):

```
SELECT last_name, department_id
FROM workers
```

Вывод:

last_name	department_id
Иванов	90

Пример запроса (2):

```
SELECT salary
FROM workers
```

Вывод:

salary
50000
40000
50000 [incorrect!!!]

Данный вывод **некорректный**, так как содержит дубликаты, исправленный запрос:

```
SELECT DISTINCT salary
FROM workers
```

Вывод:

salary
50000
40000

Пример запроса (3):

```
SELECT * FROM workers  
WHERE worker_id < 103
```

-- * => выбирает все столбцы

Пример запроса 4:

```
имя столбца [NOT LIKE / LIKE] <шаблон>
```

| Шаблоны **регистрозависимы!**

Проверка попадания в диапазон

Для проверки попадания числа в некоторый диапазон используют специальный оператор BETWEEN (или NOT BETWEEN)

Синтаксис оператора BETWEEN

```
имя столбца [BETWEEN \ NOT BETWEEN] min_val and max_val
```

| границы min и max - включаются

Проверка на попадание в диапазон

Для этого используются операторы IN и NOT IN. IN проверяет наличие значения в списке значений.

```
имя столбца [IN \ NOT IN] {знач1, знач2, ...}
```

Пример запроса:

```
SELECT last_name  
FROM workers  
WHERE department_id IN (10, 90)
```

Вывод:

last_name
Иванов
Сидоров

Работа с нулевыми значениями NULL

Пример запроса на получение не нулевых строк:

```
WHERE != NULL – некорректный запрос!!!
```

В SQL применяются два оператора для работы с NULL - **IS NULL \ IS NOT NULL**

```
SELECT last_name  
FROM workers  
WHERE salary IS NULL
```

Вывод:

last_name
Иванов

Сортировка строк

```
ORDER BY имя_столбца [ASC \ DESC]
```

По умолчанию: ASC - сортировка по возрастанию, DESC - по убыванию.

Агрегатные функции

Агрегатные функции предназначены для работы с множеством таблиц.

```
<имя функции> ([DISTINCT] название столбца | выражение)
```

Основные виды групповых операций:

- AVG - для вычисления среднего
- MAX - максимальное значение
- MIN - минимальное значение
- SUM - сумма значений строк
- COUNT (имя столбца) - подсчет строк с ненулевыми значениями
- COUNT (*) - для подсчета строк в исходном наборе (подсчет строк включает нулевые значения)

```
SELECT department_id,  
       SUM(salary) as sumsal  
FROM workers  
GROUP BY department_id
```

Вывод:

department_id	sumsal
90	10000
800	40000

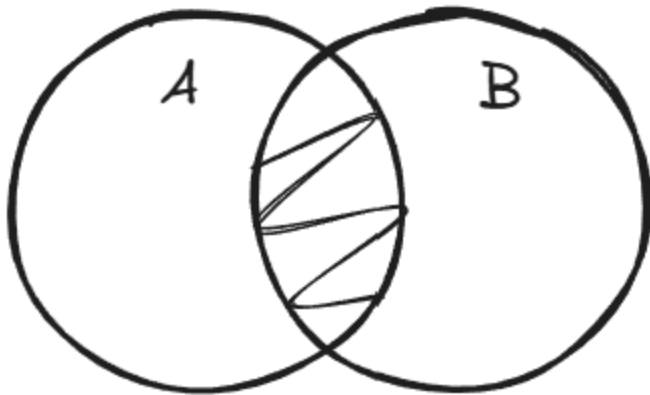
Выборка данных из нескольких таблиц

Оператор JOIN используется для выборки данных из нескольких таблиц

```
FROM <имя левой таблицы><вид соединения> JOIN <имя правой таблицы>\n[ON <условие соединения>]
```

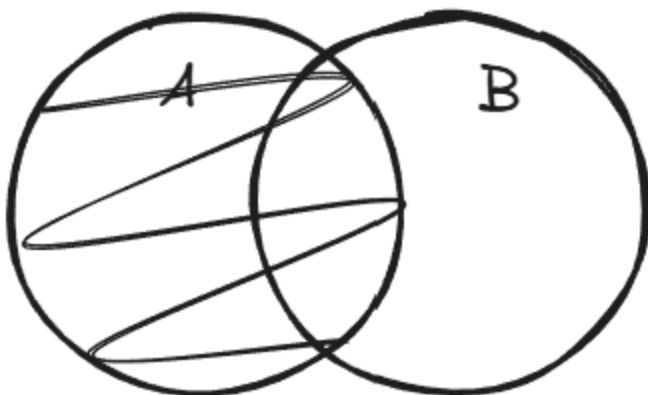
Типы соединений:

1. INNER JOIN - внутреннее соединение. Попадают только те строки таблицы которые соответствуют условию соединения



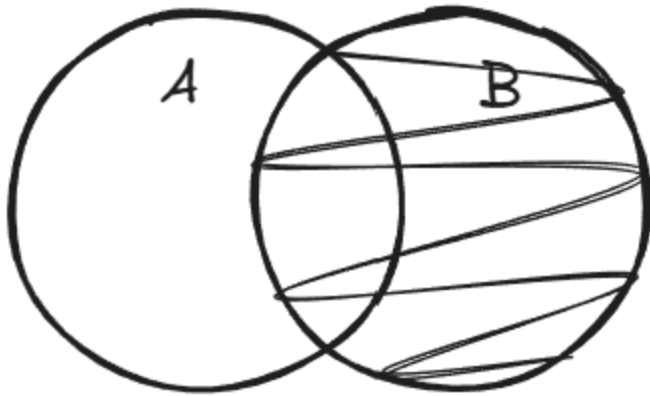
```
SELECT *  
FROM TA INNER JOIN  
TB  
ON key_A = key_B
```

2. LEFT JOIN - левое внешнее соединение. Попадут все строки записанные слева и правые удовлетворяющие условию.



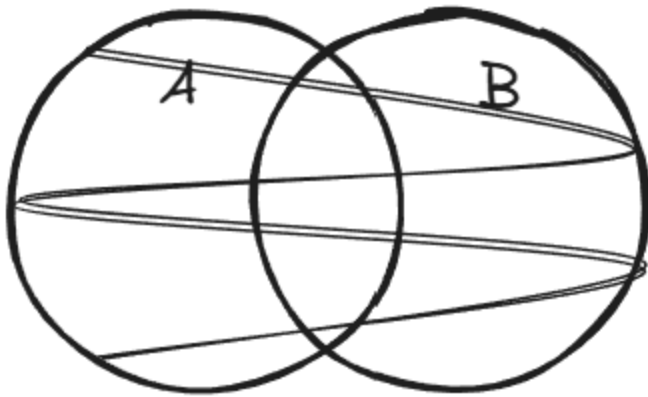
```
SELECT *  
FROM TA LEFT JOIN  
TB  
ON key_A = key_B
```

3. RIGHT JOIN - правое внешнее соединение. Попадут все строки записанные справа и левые удовлетворяющие условию.



пример SQL аналогичный

4. FULL JOIN - покрытие двух множеств (A and B)



4. Естественное соединения
5. Декартово произведение